SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	**192365013**
Name	**K.Dhilip**

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**

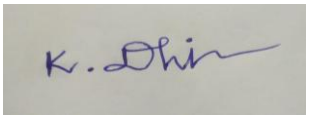
Question Count: 5

Code of Conduct

I **K.Dhilip/192365013** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1. **A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.**

The main problem is poorly configured input splits and data partitioning. Some Mappers receive much more data than others, causing data skew**.** As a result, some reducers finish early and remain idle while others continue processing.

Corrective Actions:

1. **Balance Input Splits** – Configure an appropriate block/split size so data is distributed more evenly among Mappers.
2. **Handle Data Skew** – Use a better partitioner to distribute keys evenly among Reducers.
3. **Use Combiners** – Reduce the amount of intermediate data transferred from Mappers to Reducers.
4. **Optimize Reducer Count** – Set a suitable number of Reducers based on data size and available cluster resources.
5. **Monitor the Job** – Check Mapper/Reducer execution times and identify skewed partitions for further tuning.

The failure is mainly caused by uneven data distribution (data skew)**.** Proper input splitting, balanced partitioning, and an optimized number of reducers can improve parallelism and prevent reducers from remaining idle.

2. **An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.**

Probable Design Error:
The likely problem is insufficient data replication or a single point of failure in the NameNode**.**

Analysis:

1. **Low Replication Factor:**
   If the replication factor is **1**, data exists on only one DataNode. When that node fails, the data becomes unavailable.
2. **Poor Replica Placement:**
   Replicas may be placed on the same node or rack, making them vulnerable to a single failure.
3. **Single NameNode:**
   If only one NameNode is used, its failure can make the entire HDFS namespace unavailable.

Corrective Actions:

- Set the HDFS replication factor to 3 for important data.
- Distribute replicas across different DataNodes and racks**.**
- Configure NameNode High Availability (Active/Standby)**.**
- Enable automatic block re-replication when a DataNode fails.
- Monitor DataNode and NameNode health regularly.

The issue is most likely caused by low/poorly distributed replication or a single NameNode**.** Using **3**-way replication and NameNode HA will significantly improve HDFS availability and fault tolerance.

3. **A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.**

Likely Causes:

1. **Repeated data ingestion** – NiFi may process the same file or record more than once.
2. **No unique identifier** – Records may not have a primary key or unique ID.
3. **ETL retry issues** – Failed transactions may be retried without checking whether the data was already inserted.
4. **Improper flow configuration** – NiFi processors may send the same FlowFile to multiple destinations.

Recommended Fixes:

- Use unique IDs or primary keys for every record.
- Configure NiFi with proper success/failure relationships and checkpoints**.**
- Use idempotent processing**,** so processing the same record again does not create another copy.
- Use deduplication based on record ID, file name, or checksum.
- For databases, use UPSERT/MERGE instead of blindly inserting records.
- Monitor NiFi queues and provenance data to identify repeated processing.

Duplicates mainly occur because of reprocessing, retries, or missing uniqueness checks**.** Using unique keys, deduplication, checkpoints, and idempotent ETL/NiFi flows prevents duplicate records in the analytics store.

4. **A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.**

Likely Scheduling Error:
The cluster is probably using poor queue configuration or unfair resource allocation**.** One or more users/jobs may consume most of the available CPU and memory, leaving insufficient resources for other users.

Causes:

1. No fair scheduling between users.
2. Incorrect queue capacity configuration.
3. No limits on resources per user or application.
4. Large jobs consuming excessive CPU and memory.
5. Too many jobs running simultaneously.

Recommended Fixes:

- Use YARN Capacity Scheduler or Fair Scheduler**.**
- Create separate queues for different users or workloads.
- Set minimum and maximum resource limits for each queue.
- Give important jobs higher priority**.**
- Enable resource preemption so a user cannot monopolize the cluster.
- Monitor CPU, memory, and queue utilization regularly.

Resource starvation is mainly caused by unbalanced scheduling and lack of resource limits**.** A properly configured Capacity Scheduler with queue limits, priorities, and fair resource allocation will provide better resource utilization and prevent one user from dominating the cluster

5. **A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.**

Likely Design Flaw:
The backup or replication system is using old snapshots/backups or is not properly synchronizing the latest data before failure.

Causes:

1. **Infrequent backups** – Latest changes are not included.
2. **Asynchronous replication delay** – The replica may be behind the primary storage.
3. **No backup verification** – Failed or incomplete backups may go unnoticed.
4. **Incorrect recovery point** – An older snapshot is selected during recovery.
5. **Poor backup scheduling** – Backup jobs may not run regularly.

Recommended Fixes:

- Use frequent incremental backups along with periodic full backups.
- Monitor replication lag and ensure replicas are synchronized.
- Maintain multiple recovery points**.**
- Verify backup integrity regularly through test restores.
- Use proper RPO (Recovery Point Objective) to determine how much recent data can be lost.
- Clearly identify and restore the latest valid backup/snapshot**.**

The outdated recovery is mainly caused by stale replication or an old backup/snapshot**.** Using frequent backups, synchronized replication, backup verification, and a defined RPO ensures that recovery restores the most recent valid data**.**